

Creating a web-based API for the MM-01 StudentDAQ on Raspberry Pi Hardware

Mike Bower · September 2026 · Version 1.0 · Source: github.com/mike-bower/mm01-student-daq (MIT licence)

The software described here and this paper were developed with the assistance of Claude Code, using Anthropic's Claude Opus 5 model. See Acknowledgements.

ABSTRACT

The Micro-Measurements MM01 StudentDAQ is a single-channel, USB-powered strain-gage instrument built for teaching laboratories. It ships with a Windows-only application and a .NET interface library, which ties every instrument to a Windows PC and hides the measurement chain from the student. This paper describes the design and implementation of an open-source, web-based API for the MM01 that runs on a Raspberry Pi 4 and serves a live microstrain readout, a strip chart, a REST and WebSocket interface, and CSV recording to any browser on a classroom network. The USB HID protocol was recovered from the vendor's interface library and re-implemented in about 600 lines of Python (a thousand with its documentation) over the standard `hid` binding. Because the MM01 streams conversions at a fixed 80 samples per second rather than answering polls, the server runs one reader thread per device and a separate publisher thread, with a priority lock that keeps configuration commands responsive against the continuous stream. Gage factor and balance are applied on the host, and every recording is written with a metadata sidecar so that scaled values can be audited afterwards. A pure-Python virtual device reproduces the protocol closely enough that the whole stack, from report framing to REST, is verified by 103 automated tests with no hardware attached, and students without an instrument can still complete four of the five accompanying labs. The result installs from pinned binary wheels in a few minutes, needs no build step and no internet connection after setup, and exposes the entire measurement chain as readable code.

Keywords: strain gage, data acquisition, USB HID, Raspberry Pi, FastAPI, WebSocket, laboratory teaching, open-source instrumentation

1. Objective

The MM01 is an attractive teaching instrument: it is small, powered entirely from USB, has no controls of its own, and produces a clean 24-bit conversion stream from a quarter-, half- or full-bridge strain gage. Its limitation is the software. The supplied application runs only on Windows, on the PC the instrument is plugged into, and presents the student with a finished number. Nothing in that workflow shows where the number came from, and every bench needs its own Windows machine.

The objective of this work was to replace that dependency with a small networked server on commodity hardware, so that a class can read the instrument from whatever device is already in the room. The specific goals were:

1. **Native Linux support.** Read the MM01 over USB HID with no vendor software, on a Raspberry Pi 4 running the standard 64-bit Raspberry Pi OS.
2. **Many readers, one instrument.** Serve the live reading to any number of browsers at once over HTTP and WebSocket, so an instructor's screen and every student laptop see the same stream.
3. **A complete, documented API.** Expose every instrument control (bridge configuration, gage factor, zero balance, device labelling) through a REST interface with interactive OpenAPI documentation, so that laboratory exercises can be driven from the API page as easily as from the user interface.
4. **Recording with provenance.** Write the stream to plain CSV that opens in a spreadsheet, alongside the settings that were in force, so that a recorded value can be traced back to raw counts.
5. **Code as a teaching resource.** Keep the driver short, keep one function per device operation, and make the scaling arithmetic visible, so that the labs can point at the code rather than describe it.

6. **Reliable, offline installation.** Install on the Pi with no compiler, from prebuilt wheels, and run indefinitely with no internet connection.
7. **Operation without hardware.** Provide a simulator faithful enough that the labs about scaling and zeroing can be done on any computer, and that the test suite needs no instrument.

Two things were deliberately left out. The server has no authentication, because it is meant to be reachable by every student on a closed classroom network without account management. And no attempt was made to capture every conversion with guaranteed timing; the instrument's own 80 S/s stream is preserved on the server, but the browser and the recorder sample it at configurable, slower rates, as discussed in Section 3.7.

2. Hardware and software requirements

2.1 Hardware

Table 1. Hardware used for the reference deployment.

Item	Detail
Host computer	Raspberry Pi 4 Model B, 64-bit Raspberry Pi OS (Bullseye). Any Pi that runs the 64-bit OS should work; the Pi 4 was the reference target for wheel selection and timing measurements.
Instrument	Micro-Measurements MM01 StudentDAQ. USB vendor ID 0x275F, product ID 0xF002. Single channel, 24-bit signed ADC, fixed 80 samples/second, internal completion for quarter and half bridge, USB-powered. Several units may be attached to one Pi; each is a separate device and channel.
Sensor	A foil strain gage in quarter-bridge (three-wire), half-bridge or full-bridge configuration, wired to the MM01 terminals. Lab 5 uses a gage bonded to a cantilever beam with a known mass.
Power	The official Raspberry Pi USB-C supply. The MM01 draws its power from the Pi's USB port; an under-powered Pi does not fail cleanly but drops conversions.
Cabling	A USB data cable (some cables are charge-only and carry no data).
Network	A wired or wireless classroom LAN. Clients need only a web browser. The Pi itself needs internet once, during setup, or not at all if an offline wheelhouse is prepared.

2.2 Software

The server is a FastAPI application served by Uvicorn, with the device driver written directly against the `hid` package (a ctypes binding to hidapi). Table 2 lists the pinned versions.

Table 2. Software stack. Python versions are pinned in `requirements.txt` ; every pin has a prebuilt aarch64 / CPython 3.9 wheel.

Component	Version	Role
Raspberry Pi OS (64-bit)	Bullseye	Operating system; supplies CPython 3.9 and the hidraw kernel driver
CPython	3.9	Runtime. Dictates the wheel pins and the use of <code>Optional[X]</code> in Pydantic models
libhidapi-hidraw0	apt	Native HID library the hid package loads
fastapi	0.128.8	REST routing, WebSocket endpoint, OpenAPI documentation
uvicorn[standard]	0.39.0	ASGI server (uvloop, httptools, websockets)
pydantic / pydantic-settings	2.13.5 / 2.11.0	Request and response models; <code>.env</code> configuration
hid	1.0.9	USB HID enumeration, open, read and write
pytest, pytest-asyncio, httpx	8.3.4 / 0.24.0 / 0.28.1	Test suite (development only)
Alpine.js	3.13.10	Browser reactivity for the single-page UI (vendored)
uPlot	1.6.28	Strip chart (vendored)
udev rule 99-mm01.rules	—	Grants unprivileged access to <code>/dev/hidraw*</code> for vendor 275f

The version pins are a requirement in their own right, not a convenience. Recent releases of `pydantic-core` (2.48 and later) no longer publish CPython 3.9 wheels, so an unpinned install on the Pi falls through to compiling Rust from source, which takes hours. The pinned set was resolved once, checked to be all-binary for the `manylinux2014_aarch64` / `cp39` target, and recorded with the reasoning at the top of the requirements file. A helper script downloads the same wheels on any internet-connected computer, selecting the Pi's platform rather than the host's, so that a Pi with no network can be provisioned from a USB stick:

```
pip download -d wheelhouse --python-version 3.9 --only-binary=:all: \
  --platform manylinux2014_aarch64 -r requirements-dev.txt
```

The two front-end libraries are committed to the repository rather than loaded from a CDN, for the same reason: a laboratory Pi may never see the internet again after setup, and the page must still render.

3. Methods

Figure 1 shows the finished system. The subsections below describe its parts in the order the data flows through them: the wire protocol and transport, the scaling arithmetic, the streaming and locking design inside the device manager, the HTTP interface, recording, simulation, the user interface, verification and deployment.

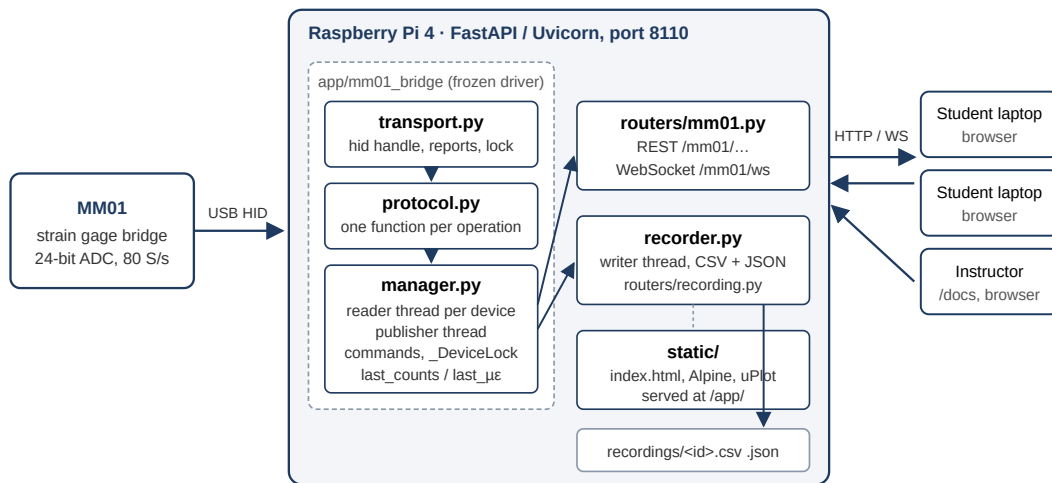


Figure 1. System architecture. The driver consumes the instrument's stream; routers and the recorder read the manager's most-recent values; browsers on the classroom network receive frames over WebSocket and issue commands over REST.

3.1 Recovering the wire protocol

No protocol description was available for the MM01. The vendor's MultiDaq distribution does, however, include `MM01InterfaceLib.dll` (version 2.0.2), a .NET assembly that wraps Silicon Labs' `SLABHIDDevice` layer. Decompiling that assembly gave the complete command set, the initialisation sequence and the scaling constants, and the driver was written to mirror it method for method so that behaviour could be compared against the vendor software.

The MM01 shares its USB vendor ID with the vendor's P3 and D4 instruments but speaks an unrelated protocol. Commands are five meaningful bytes written on the control pipe (the device has no interrupt OUT endpoint): a report ID that is always zero, a command selector the vendor calls *USBCase*, and three parameters. Conversions and command replies arrive as five-byte input reports on the interrupt IN endpoint (Table 3).

Table 3. Report formats as seen on Linux. Offsets are one lower than in the vendor's C# source because hidraw does not prepend the report-ID byte to input reports.

Direction	Bytes	Content
Host → device	0	Report ID, always 0x00
	1	USBCase: 0x01 I ² C write, 0x02 ADC control, 0x03 read version, 0x04 GPIO, 0x06/0x07 write/read ID
	2–4	Parameters p1, p2, p3; meaning depends on the USBCase
	5–16	Zero padding to the 17-byte report the descriptor declares
Device → host	0–2	24-bit signed ADC counts, big-endian, bit 23 is the sign
	3–4	Firmware major and minor version (reply to read version only)

Two details of the wire format only became apparent on Linux. First, the vendor library always sends a 64-byte buffer and lets the Windows HID layer truncate it; the Linux kernel rejects a write longer than the declared report with `EPIPE`, so the driver sends exactly 17 bytes. Second, because hidraw strips the report ID from input reports, every input offset in the recovered constants had to be reduced by one. Both facts are recorded in the constants module, which serves as the reference for the wire format.

The power-on initialisation is reproduced verbatim from the vendor's `Initdevice`: the ADC configuration register is written five times with input-buffer flushes interleaved, followed by the sample-rate command, the bridge excitation setting and the gain. The repetition looked redundant but was kept, because the ADC does not reliably latch its configuration on the first write. The sample-rate command accepts one code, which selects 80 samples per second; there is no other rate. Gain is set by writing both wipers of a digital potentiometer over I²C, and the vendor library writes the same

wiper value regardless of bridge type. Bridge selection is a single GPIO command that energises the internal completion resistors for quarter and half bridge and de-energises them for full bridge.

3.2 Transport layer

The transport module wraps one open HID handle in a small thread-safe object with four operations: `write_command`, `read_report` with a timeout, `command_read`, and `flush_input`. The write and the read of `command_read` happen under a single lock acquisition, so that a concurrent caller cannot consume the reply to somebody else's command. The flush exists because the MM01 streams continuously while its ADC is running: a freshly opened handle, or one whose ADC was left running by a previous process, has conversions queued in the kernel, and a command reply read without flushing would return a measurement instead of the reply. Enumeration filters on the vendor and product ID and sorts the result by serial number, mirroring the vendor's `GetOrderedDeviceIndex`, so that device 0 stays device 0 across a re-plug even though the hidraw path changes.

Access to `/dev/hidraw*` is root-only by default. A one-line udev rule grants mode `0666` to any hidraw device with the Micro-Measurements vendor ID, and the setup script installs it and reloads udev. The first entry in the troubleshooting guide, "the app says no device but lsusb shows it", is answered by that rule and by the fact that permissions are assigned when a device is plugged in, so it must be re-plugged after the rule is installed.

3.3 Scaling and host-side calibration

The instrument delivers raw counts; every engineering unit is computed on the host, exactly as the vendor library does it. The ADC spans ± 2.048 V over its 24-bit signed range, the sign is inverted to match the instrument's polarity, and the stored balance offset is subtracted:

$$V = -N \times 2.048 / 2^{23} - V_{\text{zero}} \quad (1)$$

Microstrain follows from the amplifier gain (100), an instrument constant (80), and the gage factor GF ; the bridge output in mV/V is a fixed multiple of the same voltage:

$$\mu\epsilon = V \times 100 \times 80 \times (2 / GF) \quad mV/V = 4 V \quad (2)$$

Neither the gage factor nor the zero offset is written to the device; the MM01 stores neither. Changing the gage factor merely re-scales subsequent readings. Zero balance averages 50 conversions and stores the result as V_{zero} in volts, so that it remains valid if the gage factor is changed later. The vendor's averaging routine flushes the input, sets the sample rate, starts the ADC, reads the requested number of conversions, waits 100 ms and then *stops* the ADC. Reproducing it faithfully therefore means the zero command must restart the ADC afterwards or the live stream dies; the driver does so in a `finally` clause, and a dedicated test asserts that the stream is still running after a zero. That the arithmetic is host-side is not an implementation detail hidden from students: it is the subject of two of the five labs, which have the student change the gage factor and watch the displayed strain scale by exactly the predicted ratio.

3.4 Streaming architecture

The defining property of the MM01 is that it streams. Once the ADC is started the device pushes a conversion every 12.5 ms and answers no read command, so the familiar design of polling the instrument once per user-interface tick is wrong: each poll would return the oldest report in the kernel buffer, and the displayed value would fall progressively further behind real time. The device manager therefore separates consumption of the stream from publication to clients (Figure 1, centre).

Each connected device gets a dedicated **reader thread** that blocks on the HID handle, decodes each report as it arrives, applies equations (1) and (2), and overwrites a per-device record of the latest counts, microstrain and mV/V. A single **publisher thread** wakes at the configured interval (200 ms by default), takes a snapshot of every device's latest values into a frame, and hands the frame to each WebSocket subscriber's queue through the event loop's thread-safe `call_soon_threadsafe`. The WebSocket handler is a plain `await queue.get()` loop that serialises the frame to JSON. The REST readings endpoint returns the same snapshot without touching the hardware at all.

The consequence, which is stated in the configuration file and the labs because it matters to the experiment, is that the publication interval controls only how often the browser is refreshed. It never changes the conversion rate, and the server always holds the newest conversion regardless of how many clients are connected or how slowly they consume frames.

3.5 Command priority under a continuous stream

The reader thread holds the per-device lock for the duration of one read, releases it, and immediately re-acquires it for the next. Commands such as changing the bridge or zeroing need the same lock for a whole multi-report sequence. With a plain mutex this arrangement fails in practice: CPython locks are not first-in-first-out, the reader wins the race for the lock almost every time, and a command thread can wait indefinitely. On hardware this appeared as bridge changes stalling for over a second, and with a silent device the command never completed at all.

The fix is a small lock that gives commands priority. A command registers itself as a waiter before it blocks, and the reader checks for waiters before each acquisition and yields while any exist. A command therefore waits for at most one in-flight read (Figure 2). That makes the reader's read timeout the worst-case addition to command latency, so it is deliberately short: 100 ms, which still tolerates eight missed conversion slots at 80 S/s before the reader wakes up empty-handed and simply loops again.

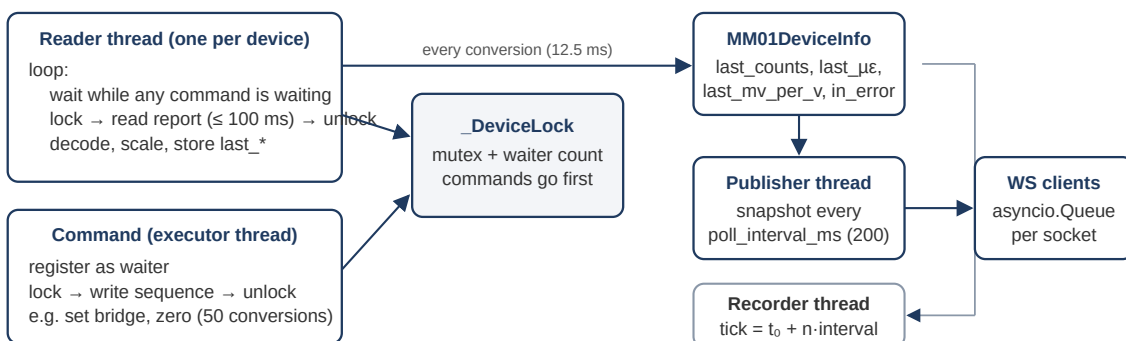


Figure 2. Thread model inside the device manager. The reader and the command paths contend for the per-device lock; the lock's waiter count lets a command pre-empt the reader after at most one read. Publisher and recorder only read the stored latest values and never touch the hardware.

3.6 REST and WebSocket interface

All device I/O is synchronous, so every route that touches hardware dispatches to the default thread pool with `run_in_executor` and the event loop is never blocked by a USB read. Requests and responses are Pydantic models, which gives validation (gage factor in (0, 9.999], label at most 32 characters, bridge as an enumeration) and an OpenAPI schema for free; the interactive documentation page at `/docs` is used directly in the labs. Readings that are not yet available are NaN inside the driver and are mapped to JSON `null` at the boundary, since NaN is not valid JSON. If the device manager failed to start, every route returns 503 with a diagnostic message; an unknown device index returns 404. Table 4 summarises the surface.

Table 4. HTTP and WebSocket endpoints. `{dev}` is the device index, which for the single-channel MM01 is also the channel index.

Method	Path	Purpose
GET	/mm01/devices	List connected instruments with serial, firmware, bridge, gage factor, zero offset
POST	/mm01/scan	Re-enumerate USB; closes and re-opens every handle
GET	/mm01/devices/{dev}	One instrument's configuration
GET	/mm01/readings	Latest microstrain and mV/V from every instrument (no hardware access)
GET	/mm01/devices/{dev}/reading	Latest reading for one instrument, including raw counts
POST	/mm01/devices/{dev}/bridge	Quarter, half or full bridge; switches internal completion
POST	/mm01/devices/{dev}/gage-factor	Set the host-side gage factor
POST	/mm01/devices/{dev}/zero	Average 50 conversions and store the balance offset in volts
DELETE	/mm01/devices/{dev}/zero	Clear the balance offset
POST	/mm01/devices/{dev}/label	Set a display label
WS	/mm01/ws	Stream of snapshot frames at the publication interval
GET	/recording/status	Whether a recording is running, and its row count so far
POST	/recording/start	Begin a recording: name, note, row interval, optional device subset
POST	/recording/stop	Close the file and return the finished session
GET	/recording/sessions	Saved recordings, newest first, with their metadata
GET	/recording/sessions/{id}/download	The CSV file
DELETE	/recording/sessions/{id}	Delete a recording and its metadata

A WebSocket frame is small and self-describing, so a student can consume it from a few lines of Python as easily as from the supplied page:

```
{
  "timestamp": 1234.567,
  "readings": {"0": 78.52},
  "mv_per_v": {"0": 0.039112},
  "error_devices": []
}
```

3.7 Recording

The MM01 converts 80 times a second and the browser sees perhaps five of those, so anything a student wants to keep has to be written down on the server. The recorder is a single writer thread that samples each selected device's latest values at a fixed interval and appends a row to a CSV file. Its tick is anchored to the start time (the n -th row is due at $t_0 + n \times \text{interval}$) rather than sleeping a fixed interval after each row, so timestamps do not drift over an hour-long recording. The interval is clamped to 10 ms to 60 s. At 12 ms the recorder captures approximately every conversion — measured on hardware at 83.1 rows per second with 7 % of rows repeating the previous conversion — while at the 50 ms default it keeps one conversion in each window and discards the rest.

That last point is a design decision that is documented rather than hidden. The recorder samples, it does not average, and the raw `counts` column is written beside the scaled values precisely so that a tick which landed between two conversions shows up as a repeated count in the data instead of being disguised. A blank cell, rather than the string "nan" or a stale value, marks an instant at which a device had no reading, because a blank is what both Excel and pandas read as missing data. The file is flushed once a second so it can be inspected while the recording is in progress.

Table 5. Columns of a recording with one device. Each additional device adds three more columns with its own index.

Column	Content
t_s	Seconds since the recording started, to the millisecond; the natural x-axis
iso_time	Local wall-clock time of the row
microstrain_0	Scaled microstrain, under the gage factor and zero offset in the sidecar
mv_per_v_0	Bridge output in mV/V
counts_0	Raw 24-bit ADC counts the two scaled values were computed from

Because gage factor, bridge and zero offset are host-side scaling, a CSV of microstrain is meaningless without them. Every session therefore produces two files: the CSV, kept free of any preamble so it opens cleanly in a spreadsheet, and a JSON sidecar carrying the name, note, start and stop times, row interval, and for each recorded device its serial number, firmware, bridge, gage factor, zero offset and whether it was simulated. The sidecar is written when the recording starts and rewritten when it stops. If the Pi loses power mid-recording the CSV survives up to the last flush and the session is still listed, marked *interrupted*, instead of vanishing. A recording stops itself after a configurable maximum (one hour by default) so that one left running overnight cannot fill the SD card; at 20 rows a second a single channel writes about 4 MB an hour.

Session identifiers are generated on the server from a timestamp and a slug of the user's name, but they come back from clients as path parameters. Every lookup matches the identifier against a strict pattern before it is joined to the recordings directory, which closes the path-traversal hole that a download endpoint otherwise opens. Only one recording runs at a time; a second start request is refused with a clear message rather than queued.

3.8 Simulation

A virtual device implements the same call surface as the real transport handle: it accepts the same five-byte commands, keeps the same state (ADC running or stopped, sample rate set, excitation on or off, wiper writes), replies to the version query in the same report layout, and returns nothing on a read while the ADC is stopped, exactly as hardware does. Its signal is a $\pm 1000 \mu\epsilon$ sine with a ten-second period and $5 \mu\epsilon$ of Gaussian noise, and it is encoded back into 24-bit counts through the inverse of equations (1) and (2) using the real scaling constants. A reader that decodes the simulated report with the production code therefore recovers the intended microstrain, and reads are paced at 12.5 ms so that timing-sensitive logic sees realistic behaviour.

The simulated device manager overrides one method, enumeration, and inherits the reader threads, publisher, lock and every command unchanged. This is what makes the simulator trustworthy as a test double: the only code it bypasses is the USB layer itself. In the user interface a prominent SIMULATOR badge is shown so that generated numbers are never mistaken for measurements, and the recording sidecar flags simulated devices for the same reason.

3.9 User interface

The front end is one HTML page and four plain scripts, with no build step, no bundler and no framework beyond Alpine.js for reactivity and uPlot for the strip chart. It shows a large microstrain readout with running minimum and maximum, the mV/V value, the stored offset, the bridge and gage factor in force, a 60-second rolling chart, controls for every command in Table 4, and a recording panel that lists, downloads and deletes sessions. The WebSocket client reconnects five seconds after a drop, and a status dot shows whether the stream is live. Keeping the page free of tooling was a goal in itself: a student can read every line of it, and an instructor can change a label with a text editor on the Pi.

3.10 Verification

The test suite runs entirely against the virtual device, in-process, through an ASGI transport, so it needs neither hardware nor a running server and completes in under ten seconds on a development machine. The setup script runs it as its final step, which turns "the install is healthy" into a fact rather than an assumption before anyone goes looking for a wiring fault. Table 6 summarises the coverage.

Table 6. Automated tests (103 in total, all hardware-free).

Group	Tests	What is checked
Protocol and transport	41	Report construction and padding, 24-bit sign decoding, version parsing, the initialisation sequence, bridge and gain commands, averaging, and scaling round trips through the virtual device
Device API	31	Every REST route and the WebSocket, including 503 and 404 behaviour, validation limits, that zeroing leaves the ADC streaming, and that gage factor changes re-scale readings without touching the device
Recording	31	Session lifecycle, CSV layout, blank cells for missing readings, sidecar written at start and stop, interrupted sessions listed, auto-stop, identifier validation, delete, and that the live session is excluded from the list

Two measures keep the results identical on every machine. The test configuration ignores any `.env` file and scrubs every `MM01_*` environment variable, so a student who has switched on simulator mode for a lab sees the same results as everyone else. And recording tests write only to a temporary directory, never to the real `recordings/` folder, which holds student data and is never cleaned up by anything in the repository.

Some behaviour can only be observed on hardware, and those observations shaped the design: the command starvation of Section 3.5 was measured on a real MM01 as a bridge change stalling for over a second; the `EPIPE` rejection of a 65-byte report and the queue of stale conversions on a freshly opened handle were both found on Linux. Each is recorded in a comment beside the code it justified.

3.11 Deployment and maintenance

Installation is one script. It installs two system packages, creates a virtual environment, installs the pinned Python packages (from the offline wheelhouse if one is present), installs the udev rule, downloads the two front-end libraries if they are missing, and runs the tests. A second script starts Uvicorn on port 8110, bound to every interface so that laptops can reach it; the page redirects to the application, and the API documentation is one path away. Configuration is seven environment variables with working defaults: automatic scan, publication interval, simulator on/off and count, recording directory, default row interval and maximum recording length.

The driver directory is a frozen copy from a larger instrumentation project, refreshed by a synchronisation script rather than edited in place, so that the teaching kit remains stable and driver fixes arrive with their upstream tests. The recorder was built outside that directory, against the manager's public surface only, precisely so that a synchronisation cannot remove it. That boundary is the one piece of project structure a maintainer must know, and it is written down in the repository's contributor notes.

4. Conclusion

All seven objectives of Section 1 were met. A Raspberry Pi 4 running the software reads one or more MM01 instruments natively over USB HID, serves the same live stream to every browser on the network, exposes each control through a documented REST API, records to auditable CSV, installs offline from pinned binary wheels, and runs the same code path with or without hardware. The driver, at about 600 lines of code across four modules with as much again in commentary, is small enough that the labs refer students to specific functions in it, and the scaling arithmetic they are asked to predict is the arithmetic that produces the number on the screen.

Four lessons generalise beyond this instrument:

- **Establish whether the device streams or answers.** The MM01 pushes conversions and ignores polls. Every sensible design decision in the server, from the reader threads to the meaning of the publication interval, followed from recognising that early. A polled design would have shipped and then displayed steadily staler data.
- **Lock fairness matters when one party never lets go.** A reader that re-acquires a lock the instant it releases it will starve every other user under CPython's non-FIFO locks. A waiter count is a few lines and turns an indefinite stall into a bounded one, and the read timeout then becomes a latency budget to be set deliberately.

- **Vendor offsets are platform offsets.** The report-ID byte that Windows prepends and Linux does not is the kind of detail that produces plausible garbage rather than an error. Recording the shift once, in the constants module, kept it from being rediscovered in every parser.
- **On constrained targets, pinning is a requirement.** A single upstream package dropping a Python version turned a two-minute install into a multi-hour compile. Resolving the dependency set once for the exact platform, and explaining the pins where the next maintainer will look, is cheaper than any amount of troubleshooting in a classroom.

The limitations are the ones chosen. There is no authentication, so the server belongs on a closed network. The recorder samples the latest conversion rather than capturing or averaging all of them, which is right for a hand-loaded beam and wrong for a transient shorter than the row interval; the raw counts column makes the difference visible but does not remove it. Only one recording runs at a time, and only one process may hold an instrument. Natural extensions include a ring buffer in the reader thread so that a recording can capture every conversion regardless of interval, a systemd unit so the Pi serves the page from power-on, and optional decimation with averaging for students who want a quieter number and understand what they are trading for it. None of these require changes to the protocol layer, which is the part of the work that was hardest to obtain and is now, for the first time, available in readable form.

Acknowledgements and AI assistance

This work was carried out with the assistance of **Claude Code**, Anthropic's agentic coding tool, running the **Claude Opus 5** model. That assistance covered the whole of the project and is disclosed here in the interest of transparency, because a teaching kit invites the reader to trust and reuse its code.

The model contributed to the analysis of the decompiled vendor assembly, the Python implementation of the transport, protocol, manager, recorder and web layers, the test suite, the laboratory guides, and the drafting of this paper. It is recorded as a co-author on each commit in the repository, so the extent of its involvement can be inspected in the version history rather than taken on trust.

Responsibility for the result rests with the author. Every design decision described in Section 3 was reviewed and accepted deliberately, and the claims made here were checked rather than assumed: the 103 automated tests of Section 3.10 run against a virtual device that mirrors the real protocol, and the behaviour that only hardware can show — command starvation under the continuous stream, the rejection of an over-long report, the queue of stale conversions on a freshly opened handle, and the recorded throughput at a 12 ms interval — was measured on a physical MM01 attached to a Raspberry Pi. Where an observation came from hardware it is identified as such in the text.

Readers reproducing this work should note that the protocol constants originate from a decompiled vendor library and are validated by the tests and by agreement with the vendor software, not by a published specification.

References

1. Micro-Measurements (Vishay Precision Group). *MM01 StudentDAQ / MultiDaq* software distribution, including `MM01InterfaceLib.dll` v2.0.2. Source of the recovered protocol and scaling constants.
2. M. Bower. *MM01 StudentDAQ — Raspberry Pi kit*. Source code, laboratory guides and this paper: github.com/mike-bower/mm01-student-daq. MIT licence.
3. S. Ramírez. *FastAPI*. fastapi.tiangolo.com.
4. libusb project. *hidapi*, and the Python `hid` binding. github.com/libusb/hidapi.
5. C. Porzio. *Alpine.js*. alpinejs.dev.
6. L. Oniya. *uPlot*. github.com/leeoniya/uPlot.
7. Anthropic. *Claude Code* and the *Claude Opus 5* model. claude.com/claude-code. Used as described under Acknowledgements.

Creating a web-based API for the MM-01 StudentDAQ on Raspberry Pi Hardware · M. Bower, September 2026 · Released under the MIT licence with the accompanying source code. · Developed with Claude Code (Claude Opus 5).